Monoid & Semigroup

Season: 6 · FP Foundations · Category: Functional · Level: Intermediate

Intent

- Associative combine with (optional) empty.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Addition of numbers; concatenation of strings.

When to Use

- Pure functional style in Java 25 + Zazr.
- The design force named in Intent shows up more than once.

When Not to Use

- Throwing exceptions everywhere is 'fine' for a script.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.functional.monoid;

import java.util.List;

interface Semigroup<A> {
    A combine(A a, A b);
}

public interface Monoid<A> extends Semigroup<A> {
    A empty();

    default A fold(List<A> xs) {
        A acc = empty();
        for (A x : xs) acc = combine(acc, x);
        return acc;
    }
}
```

Modern Java 25 / Functional

- Primary examples use Zazr; VAVR shown in comparison episode.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: functional/monoid/
```

Pitfalls

- Zazr rejects null inside Option/collections.
- Keep interpreters at the edge.

Interview Q&A

- Q: Interview: what problem does Monoid & Semigroup solve? A: Associative combine with (optional) empty.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Associative combine with (optional) empty.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Addition of numbers; concatenation of strings. [0:40] Intent — State the design force in one sentenc...